product at the completion of the program. Line 011 controls the 20 loops needed for the calculation, using the variable *n* as a loop counter. The only statement within the loop is line 012, which multiplies the corresponding coefficients from the two arrays, and adds the product to the accumulator variable, *s*. (If you are not familiar with C, the statement: $s += x[n] * y[n]$ means the same as: $s = s + x[n] * y[n]$). After the loop, the value in the accumulator, *s*, is transferred to the output variable, *result*, in line 013.

A key advantage of using a high-level language (such as C, Fortran, or Basic) is that the programmer does not need to understand the architecture of the microprocessor being used; knowledge of the architecture is left to the compiler. For instance, this short C program uses several variables: *n*, *s*, *result*, plus the arrays: *x*[ ] and *y*[ ]. All of these variables must be assigned a "home" in hardware to keep track of their value. Depending on the microprocessor, these storage locations can be the general purpose data registers, locations in the main memory, or special registers dedicated to particular functions. However, the person writing a high-level program knows little or nothing about this memory management; this task has been delegated to the software engineer who wrote the compiler. The problem is, these two people have never met; they only communicate through a set of predefined rules. High-level languages are easier than assembly because you give half the work to someone else. However, they are less efficient because you aren't quite sure how the delegated work is being carried out.

In comparison, Table 28-3 shows the dot product program written in assembly for the SHARC DSP. The assembly language for the Analog Devices DSPs (both their 16 bit fixed-point and 32 bit SHARC devices) are known for their simple algebraic-like syntax. While we won't go through all the details, here is the general operation. Notice that *everything* relates to hardware; there are no abstract variables in this code, only data registers and memory locations.

Each semicolon represents a clock cycle. The arrays *x*[ ] and *y*[ ] are held in circular buffers in the main memory. In lines 001 and 002, registers i4

```
001   i12 = _y;                          /* i12 points to beginning of y[ ] */
002   i4 = _x;                           /* i4 points to beginning of x[ ] */
003
004   lcntr = 20, do (pc,4) until lce;   /* loop for the 20 array entries */
005     f2 = dm(i4,m6);                  /* load the x[ ] value into register f2 */
006     f4 = pm(i12,m14);                /* load the y[ ] value into register f4 */
007     f8 = f2*f4;                      /* multiply the two values, store in f8 */
008     f12 = f8 + f12;                  /* add the product to the accumulator in f12 */
009
010   dm(_result) = f12;                 /* write the accumulator to memory */
```

TABLE 28-3
Dot product in assembly (unoptimized). This program calculates the dot product of the two arrays, x[ ] and y[ ], and stores the result in the variable, *result*. This is assembly code for the Analog Devices SHARC DSPs. See the text for details.